Course: Cryptography and Network Security
Code: CSN15403
Branch: B.Tech -5th Semester

Lecture – 9: Advanced Encryption Standard (AES)

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

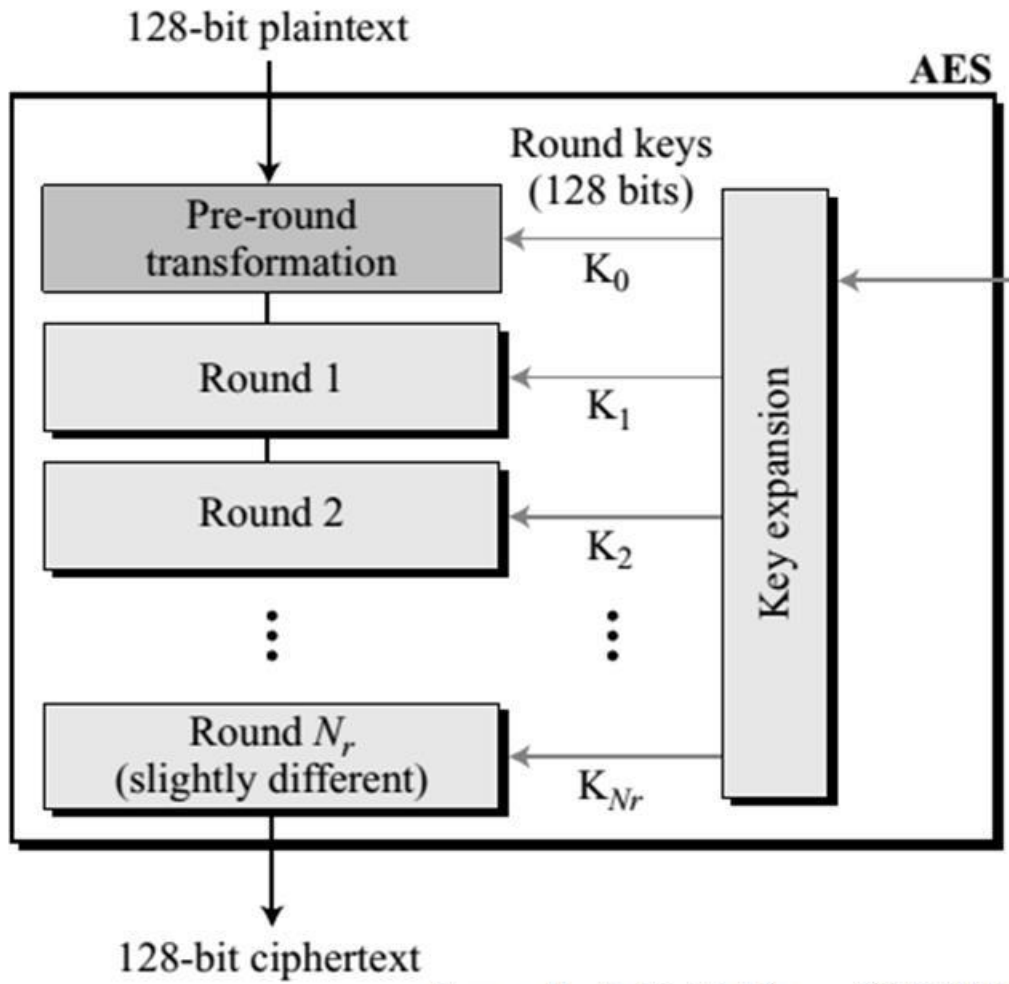
Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

INTRODUCTION

- The Advanced Encryption Standard (AES) is a symmetric-key block cipher published by the National Institute of Standards and Technology (NIST) in December 2001.
- The criteria defined by NIST for selecting AES fall into three areas: security, cost, and implementation.
- AES has defined three versions, with 10, 12, and 14 rounds.
- Each version uses a different cipher key size (128, 192, or 256), but the round keys are always 128 bits.

General design of AES encryption cipher



Cipher key
(128, 192, or 256 bits)

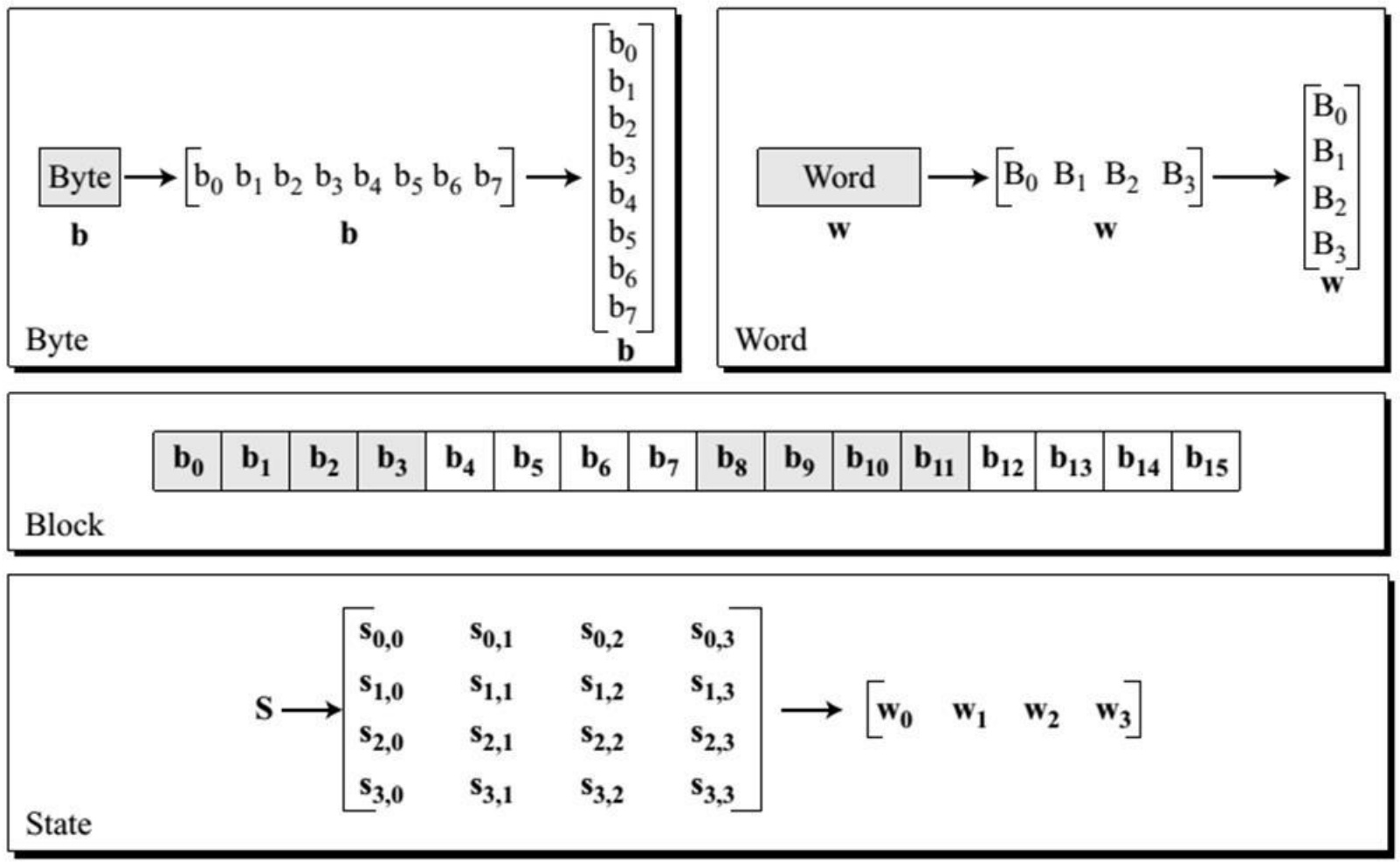
Nr	Key size
10	128
12	192
14	256

Relationship between number of rounds and cipher key size

Number of round keys = $N_r + 1$

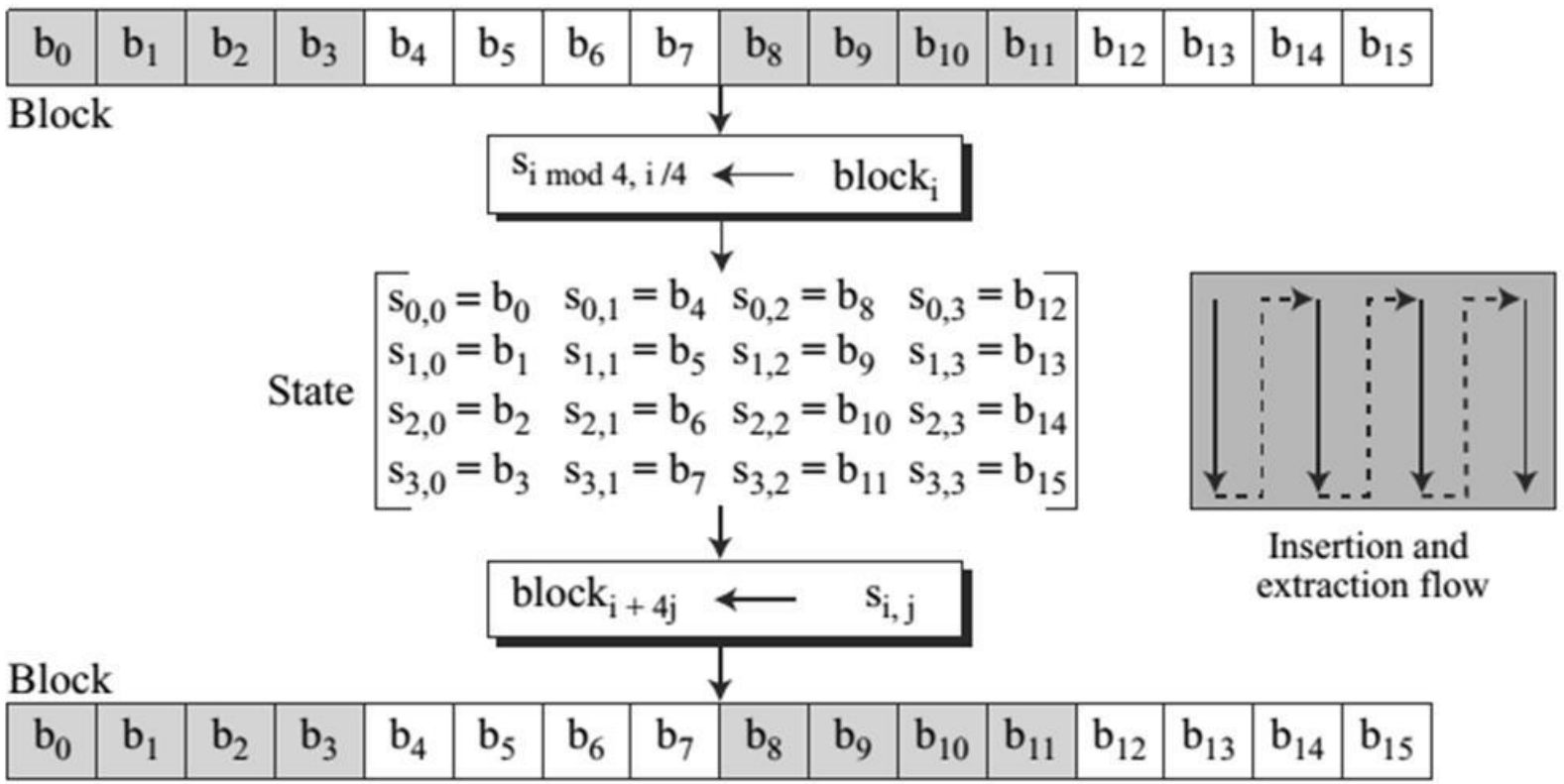
Data Units

- AES uses five units of measurement to refer to data: bits, bytes, words, blocks, and state.
- The bit is the smallest and atomic unit; other units can be expressed in terms of smaller ones.



Block-to-state and state-to-block transformation

- Although the states in different stages are normally called S, we occasionally use the letter T to refer to a temporary state.



Example #1

- Assume that the text block is “AES uses a matrix”.
- We add two bogus characters at the end to get “AESUSESAMATRIXZZ’

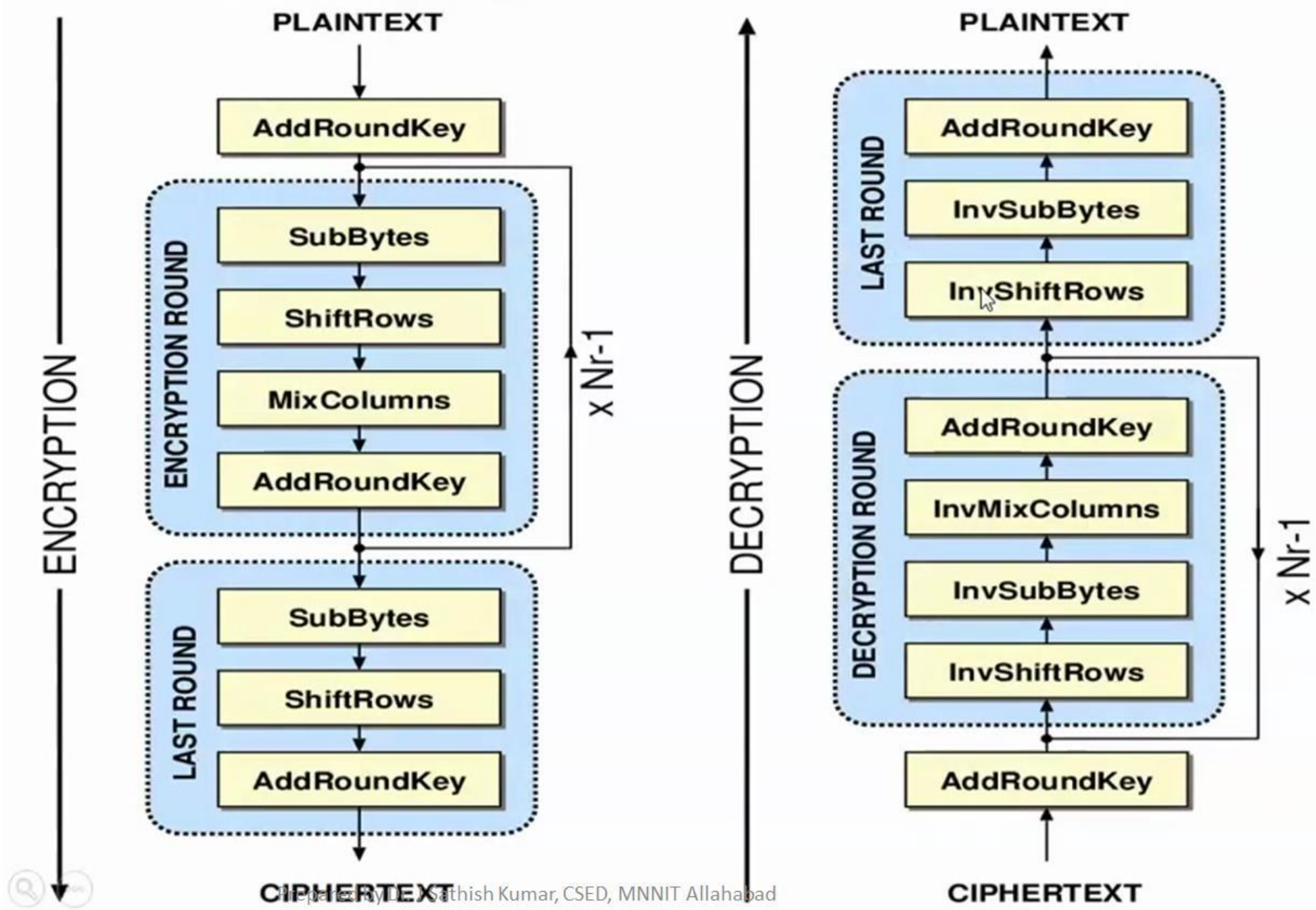
e

Text	A	E	S	U	S	E	S	A	M	A	T	R	I	X	Z	Z
Hexadecimal	00	04	12	14	12	04	12	00	0C	00	13	11	08	23	19	19

$$\begin{bmatrix} 00 & 12 & 0C & 08 \\ 04 & 04 & 00 & 23 \\ 12 & 12 & 13 & 19 \\ 14 & 00 & 11 & 19 \end{bmatrix}$$ State

	DEC	HEX		DEC	HEX
A	00	00	N	13	0D
B	01	01	O	14	0E
C	02	02	P	15	0F
D	03	03	Q	16	10
E	04	04	R	17	11
F	05	05	S	18	12
G	06	06	T	19	13
H	07	07	U	20	14
I	08	08	V	21	15
J	09	09	W	22	16
K	10	0A	X	23	17
L	11	0B	Y	24	18
M	12	0C	Z	25	19

AES


ENCRYPTION
PLAINTEXT
AddRoundKey
ENCRYPTION ROUND
SubBytes
ShiftRows
MixColumns
AddRoundKey
x Nr-1
LAST ROUND
SubBytes
ShiftRows
AddRoundKey
CIPHERTEXT
DECRYPTION
PLAINTEXT
LAST ROUND
AddRoundKey
InvSubBytes
InvShiftRows
DECRYPTION ROUND
AddRoundKey
InvMixColumns
InvSubBytes
InvShiftRows
x Nr-1
AddRoundKey
CIPHERTEXT

TRANSFORMATIONS

- To provide security, AES uses four steps for each round:
  - substitution,
  - permutation,
  - mixing, and
  - key-adding.
- The above steps are also called AES Transformation function

Substitution

- First, the substitution is done for each byte.
- Second, only one table is used for transformation of every byte, which means that if two bytes are the same, the transformation is also the same.
- Third, the transformation is defined by either a table lookup process or mathematical calculation in the GF(2^8) field.
- AES uses two invertible transformations.
  - SubBytes
  - InvSubBytes

Substitution :SubBytes

- The SubBytes operation involves 16 independent byte-to-byte transformations.
- Substitution table (S-box) for SubBytes transformation
- For example, two bytes, $5A_{16}$ and $5B_{16}$, which differ only in one bit (the rightmost bit) are transformed to BE_{16} and 39_{16}, which differ in four bits.

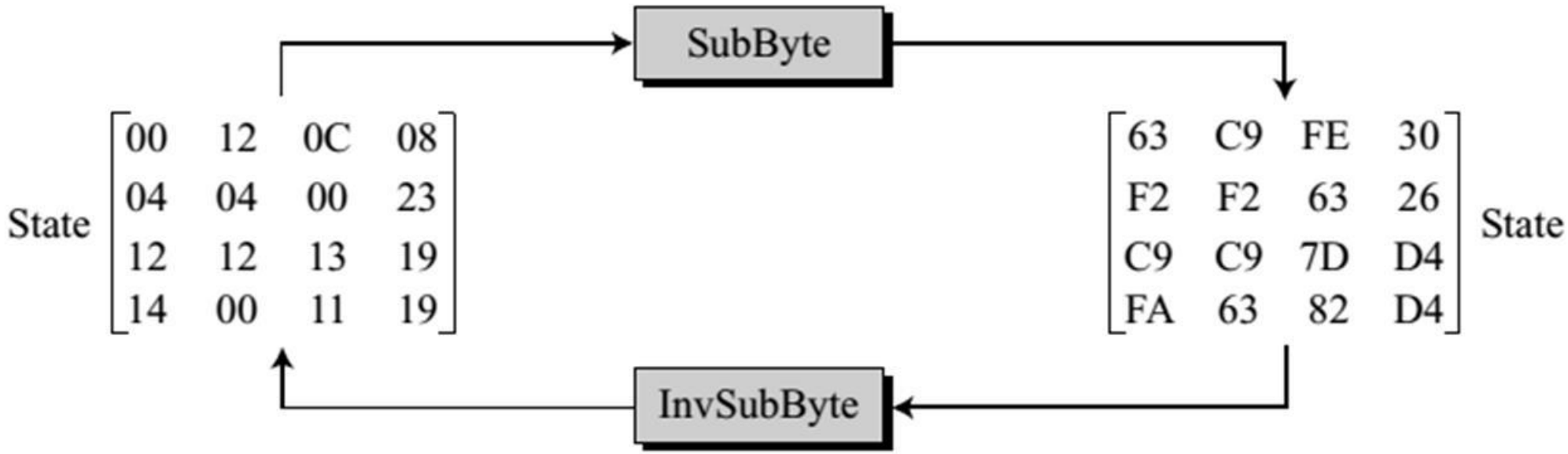
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	CB	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Substitution :InvSubBytes

- InvSubBytes is the inverse of SubBytes

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

SubBytes transformation for Example #2



If the two bytes have the same values, their transformation is also the same.
For example, the two bytes 04_{16} and 04_{16} in the left state are transformed to $F2_{16}$ and $F2_{16}$ in the right state and vice versa.

Pseudocode for SubBytes transformation

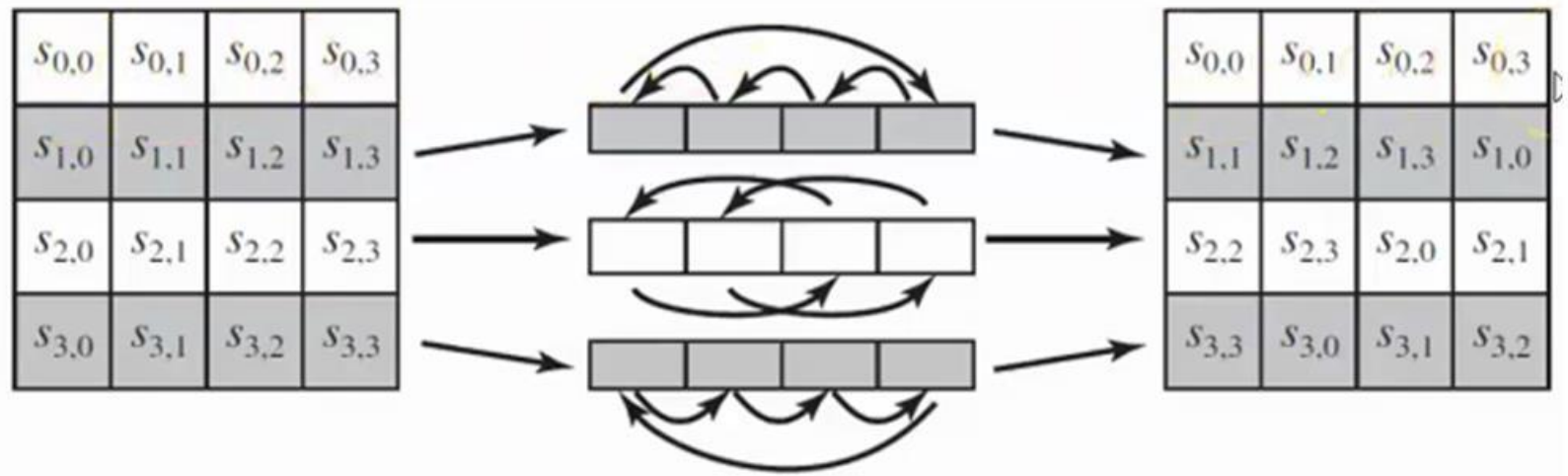
```
SubBytes (S)
{
   for (r = 0 to 3)
     for (c = 0 to 3)
           S_r,c = subbyte (S_r,c)
}
```

```
subbyte (byte)
{
   a ← byte^-1                    // Multiplicative inverse in GF(2^8) with inverse of 00 to be 00
    ByteToMatrix (a, b)
    for (i = 0 to 7)
    {
        c_i ← b_i ⊕ b_(i+4)mod 8 ⊕ b_(i+5)mod 8 ⊕ b_(i+6)mod 8 ⊕ b_(i+7)mod 8
        d_i ← c_i ⊕ ByteToMatrix (0x63)
   }
    MatrixToByte (d, d)
    byte ← d
}
```

Permutation

- Another transformation found in a round is shifting, which permutes the bytes.
- Unlike DES, in which permutation is done at the bit level, shifting transformation in AES is done at the byte level; the order of the bits in the byte is not changed.
- The ShiftRows and InvShiftRows transformations are inverses of each other

- Rules of shifting rows,
  - Row 1 → No Shifting
  - Row 2 → 1 byte left shift
  - Row 3 → 2 byte left shift
  - Row 4 → 3 byte left shift

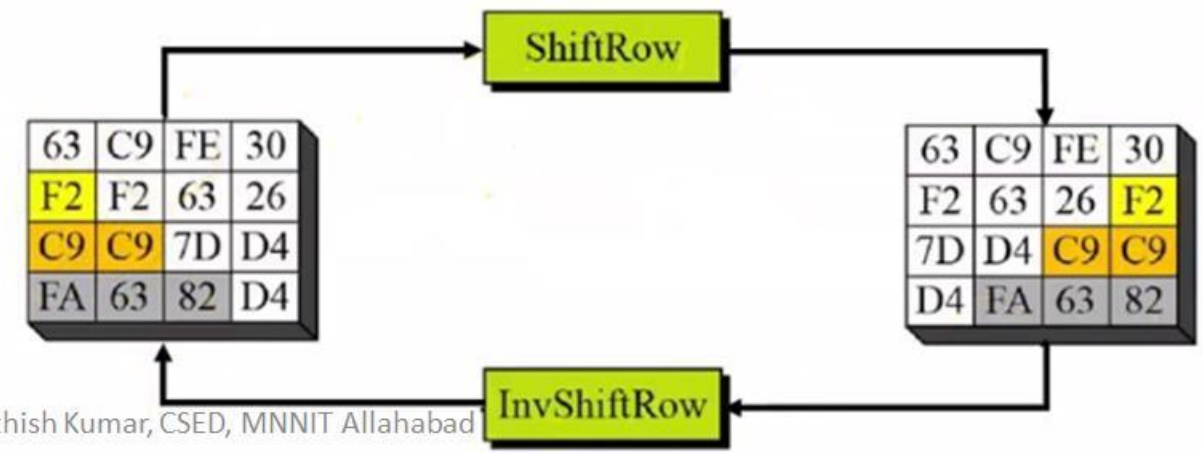


Permutation : Shift Rows

```
ShiftRows (S)
{
    for (r = 1 to 3)
        shiftrow (s_r, r)                    // s_r is the rth row
}
```

```
shiftrow (row, n)                            // n is the number of bytes to be shifted
{
   CopyRow (row, t)                          // t is a temporary row
   for (c = 0 to 3)
        row_((c – n) mod 4) ← t_c
}
```

Example #3

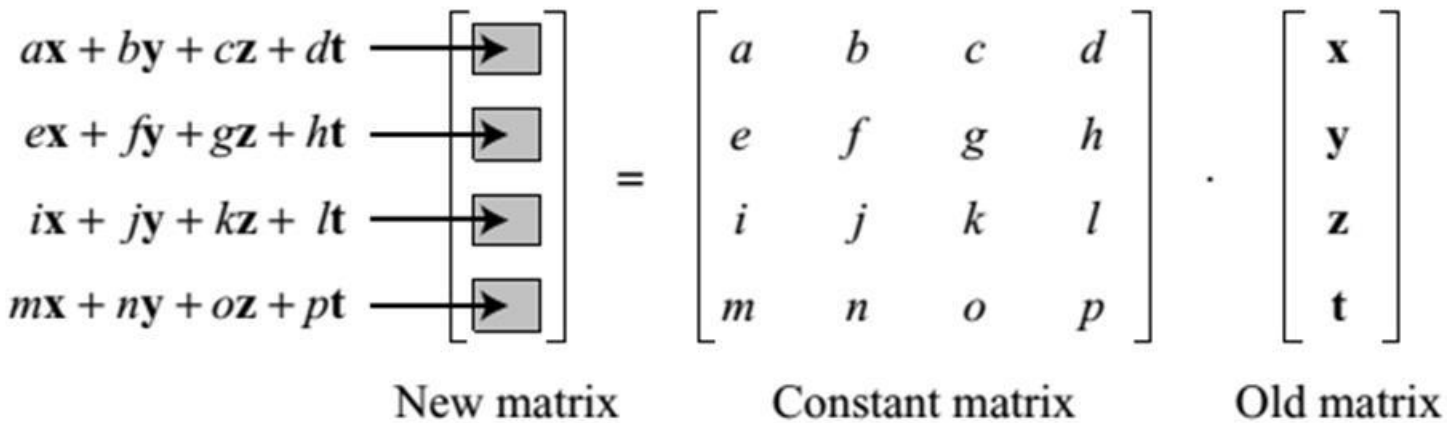


Mixing

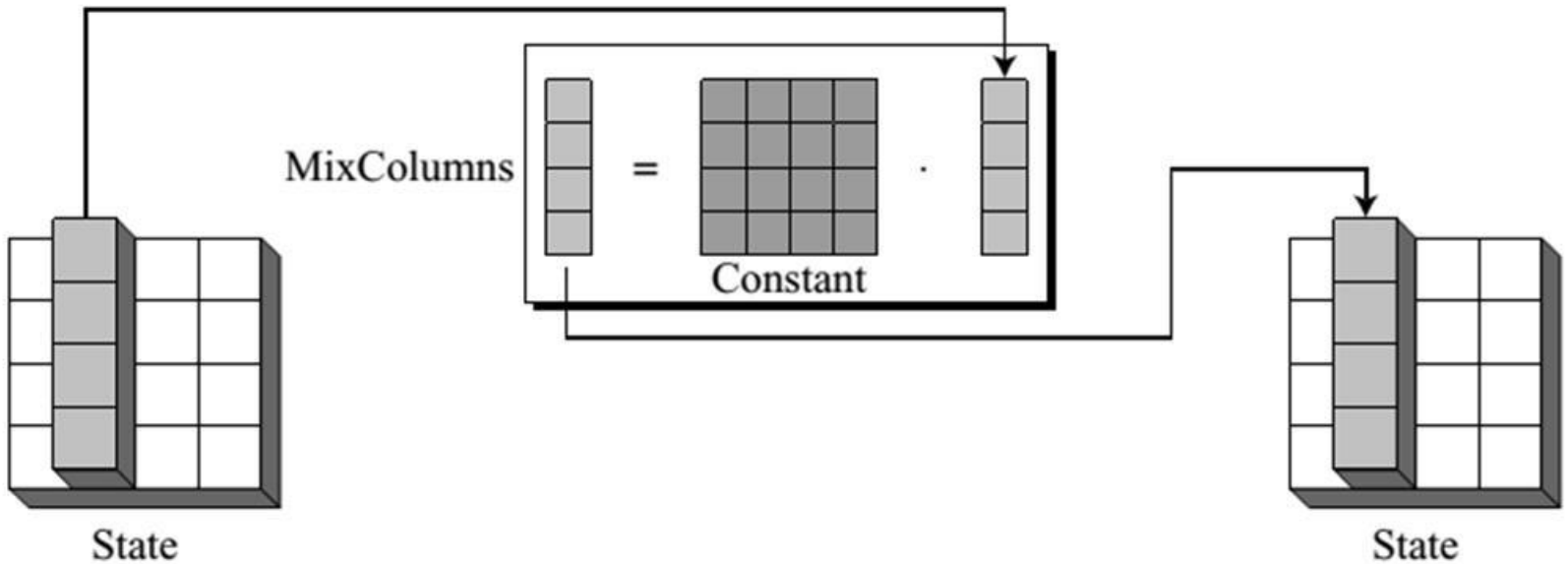
- The substitution provided by the SubBytes transformation changes the value of the byte based only on original value and an entry in the table;
  - The process does not include the neighboring bytes.
  - We can say that SubBytes is an intrabyte transformation.
- The permutation provided by the ShiftRows transformation exchanges bytes without permuting the bits inside the bytes.
  - We can say that ShiftRows is a byte-exchange transformation.
- We also need an interbyte transformation that changes the bits inside a byte, based on the bits inside the neighboring bytes.
- We need to mix bytes to provide diffusion at the bit level.

Mixing

- The mixing transformation changes the contents of each byte by taking four bytes at a time and combining them to recreate four new bytes.
- To guarantee that each new byte is different (even if all four bytes are the same), the combination process first multiplies each byte with a different constant and then mixes them.
- The mixing can be provided by matrix multiplication.



MixColumns & InvMixColumns



$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \quad \text{Inverse} \quad \begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix}$$

C $\qquad$ C^{-1}

Pseudocode for MixColumns transformation

```
MixColumns (S)
{
    for (c = 0 to 3)
        mixcolumn (s_c)
}
```

```
mixcolumn (col)
{
   CopyColumn (col, t)                    // t is a temporary column

   col_0 ← (0x02) • t_0 ⊕ (0x03 • t_1) ⊕ t_2 ⊕ t_3

   col_1 ← t_0 ⊕ (0x02) • t_1 ⊕ (0x03) • t_2 ⊕ t_3

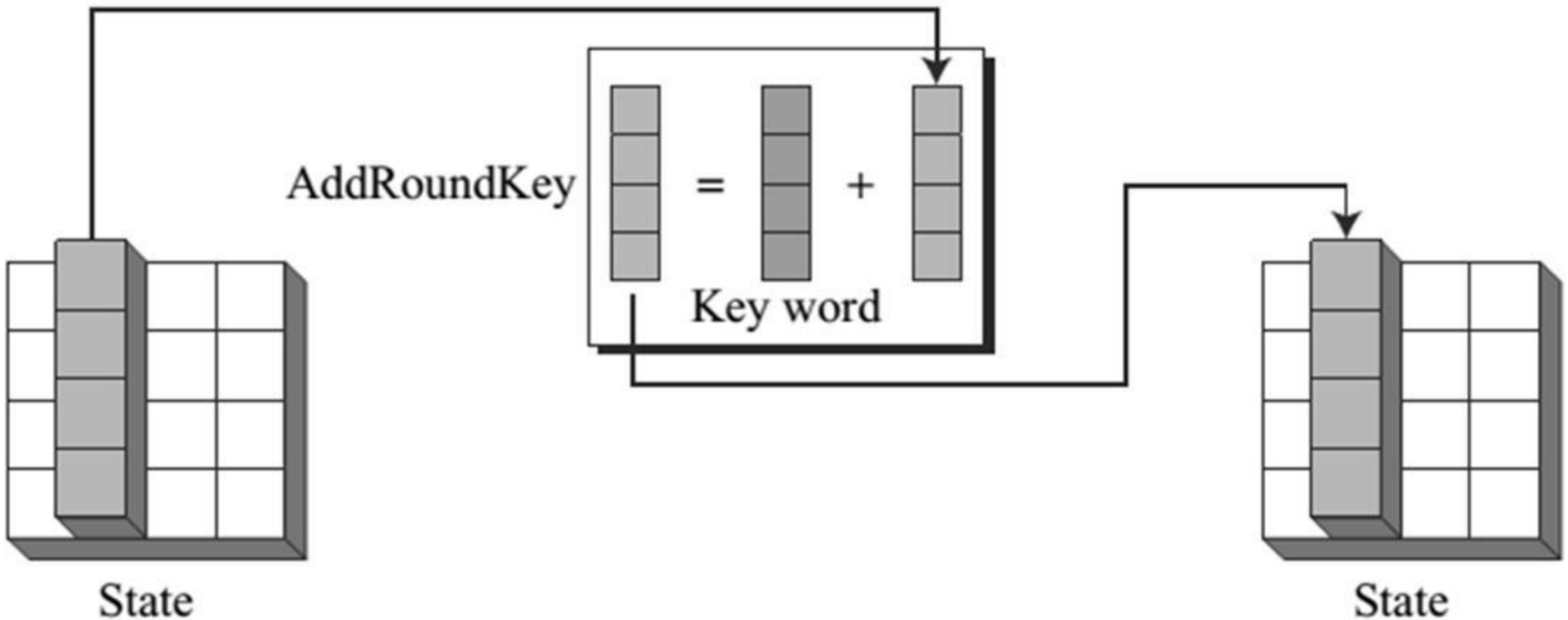
   col_2 ← t_0 ⊕ t_1 ⊕ (0x02) • t_2 ⊕ (0x03) • t_3

   col_3 ← (0x03 • t_0) ⊕ t_1 ⊕ t_2 ⊕ (0x02) • t_3
}
```

Key Adding: AddRoundKey

- AddRoundKey also proceeds one column at a time.
- It is similar to MixColumns in this respect.
- MixColumns multiplies a constant square matrix by each state column;
- AddRoundKey adds a round key word with each state column matrix.
- The operation in MixColumns is matrix multiplication; the operation in AddRoundKey is matrix addition.
- Since addition and subtraction in this field are the same, the AddRoundKey transformation is the inverse of itself.

AddRoundKey transformation



Pseudocode for AddRoundKey transformation

AddRoundKey (S)

{

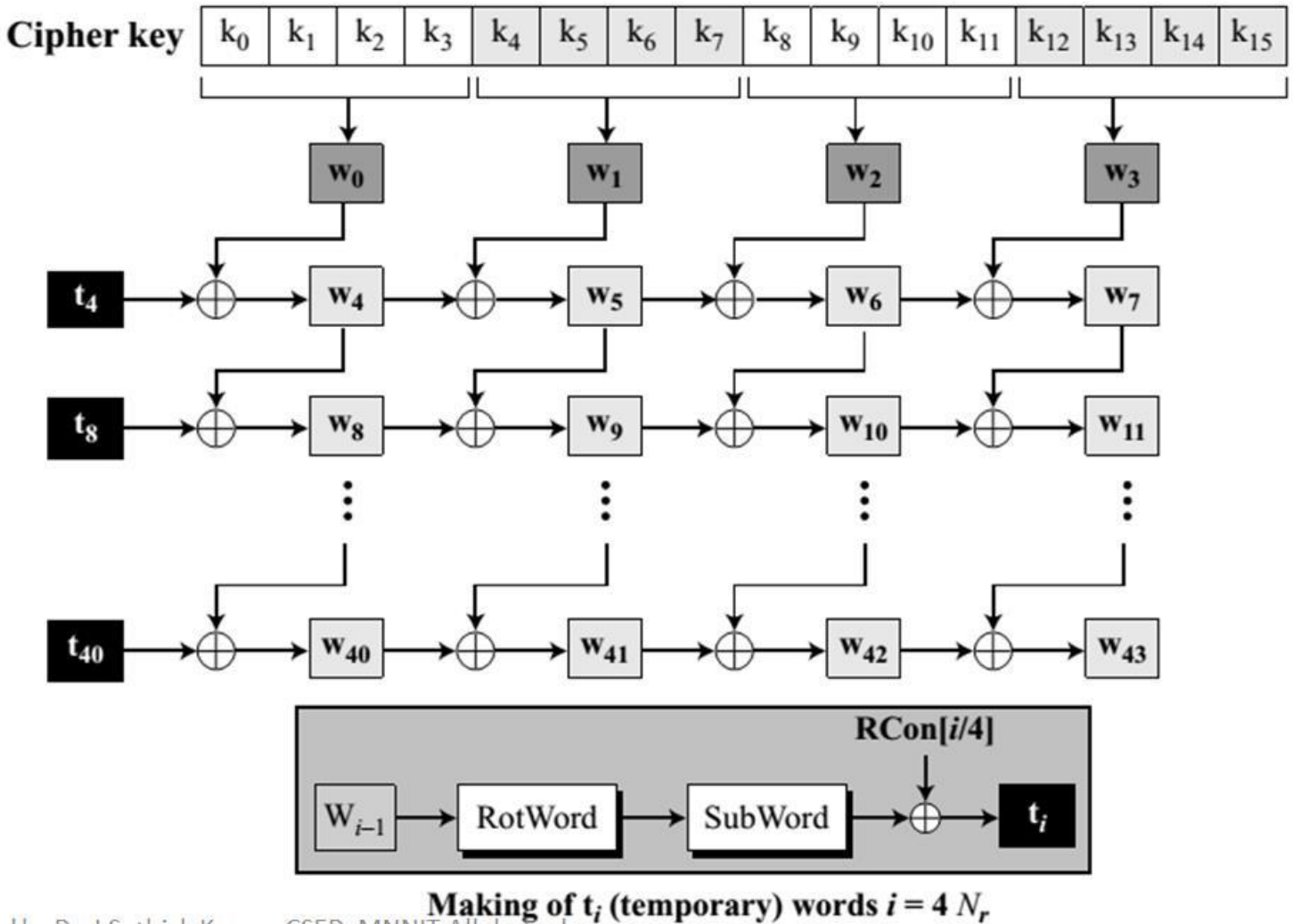
for ($c = 0$ to 3)

$\mathbf{s}_c \leftarrow \mathbf{s}_c \oplus \mathbf{w}_{4\,\text{round} + c}$

}

KEY EXPANSION

- In the AES-128 version (10 rounds), there are 44 words;
- In the AES-192 version (12 rounds), there are 52 words; and
- In the AES-256 version (with 14 rounds), there are 60 words.
- Each round key is made of four words.



Making of t_i (temporary) words $i = 4\ N_r$

KEY EXPANSION

- RotWord
  - The RotWord (rotate word) routine is similar to the ShiftRows transformation, but it is applied to only one row.
  - The routine takes a word as an array of four bytes and shifts each byte to the left with wrapping.
- SubWord
  - The SubWord (substitute word) routine is similar to the SubBytes transformation, but it is applied only to four bytes.
  - The routine takes each byte in the word and substitutes another byte for it.
- Round Constants
  - Each round constant, RCon, is a 4-byte value in which the rightmost three bytes are always zero. Table shows the values for AES-128 version (with 10 rounds).

KEY EXPANSION

Round	*Constant (RCon)*	*Round*	*Constant (RCon)*
1	$(\underline{\mathbf{01}}\ 00\ 00\ 00)_{16}$	6	$(\underline{\mathbf{20}}\ 00\ 00\ 00)_{16}$
2	$(\underline{\mathbf{02}}\ 00\ 00\ 00)_{16}$	7	$(\underline{\mathbf{40}}\ 00\ 00\ 00)_{16}$
3	$(\underline{\mathbf{04}}\ 00\ 00\ 00)_{16}$	8	$(\underline{\mathbf{80}}\ 00\ 00\ 00)_{16}$
4	$(\underline{\mathbf{08}}\ 00\ 00\ 00)_{16}$	9	$(\underline{\mathbf{1B}}\ 00\ 00\ 00)_{16}$
5	$(\underline{\mathbf{10}}\ 00\ 00\ 00)_{16}$	10	$(\underline{\mathbf{36}}\ 00\ 00\ 00)_{16}$

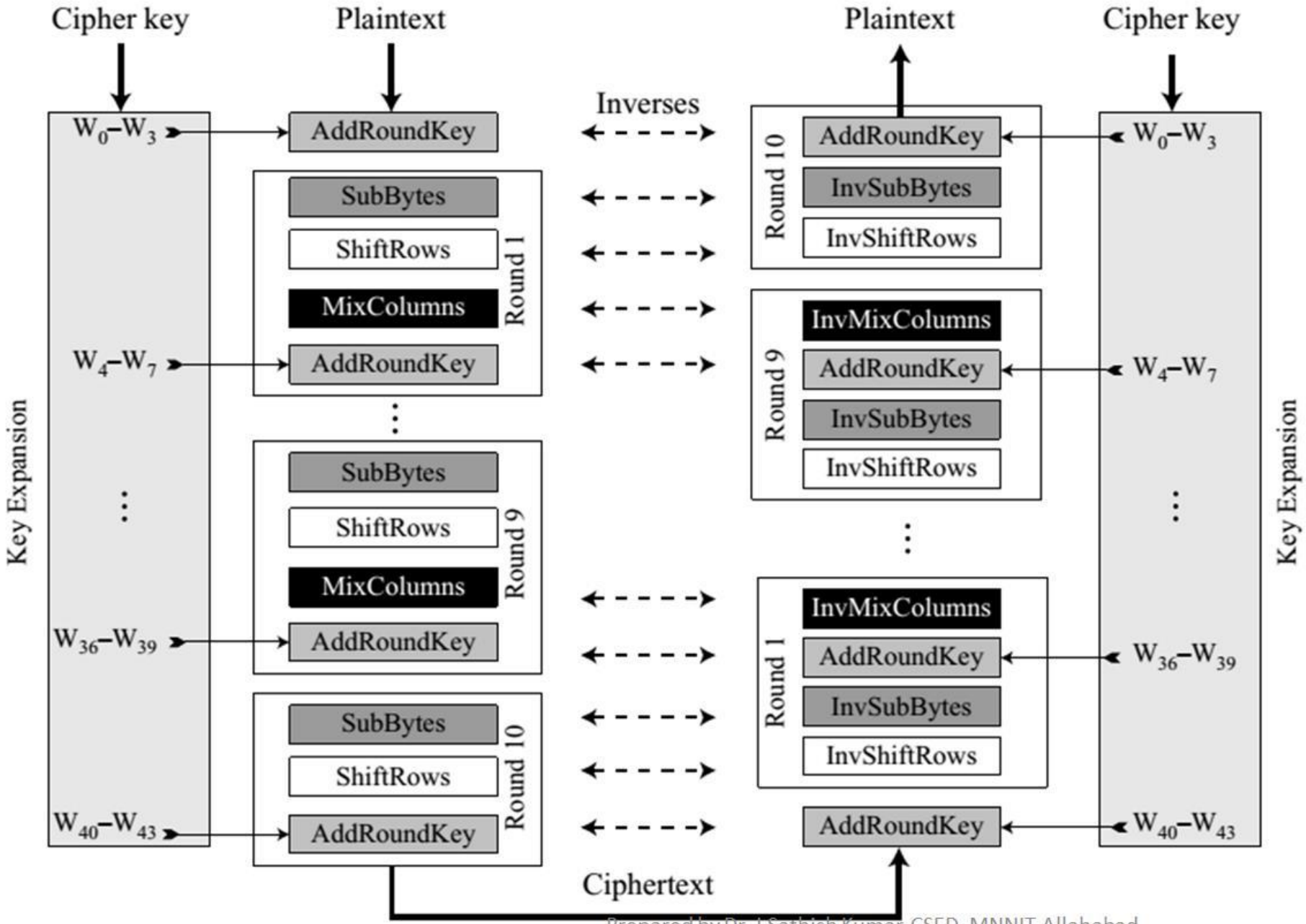
The key-expansion routine can either use the above table when calculating the words or use the GF(2^8) field to calculate the leftmost byte dynamically, as shown below (prime is the irreducible polynomial):

$$
\begin{array}{llllll}
RC_1 & \rightarrow x^{1-1} & = x^0 & \bmod prime & = 1 & \rightarrow 00000001 \rightarrow 01_{16} \\
RC_2 & \rightarrow x^{2-1} & = x^1 & \bmod prime & = x & \rightarrow 00000010 \rightarrow 02_{16} \\
RC_3 & \rightarrow x^{3-1} & = x^2 & \bmod prime & = x^2 & \rightarrow 00000100 \rightarrow 04_{16} \\
RC_4 & \rightarrow x^{4-1} & = x^3 & \bmod prime & = x^3 & \rightarrow 00001000 \rightarrow 08_{16} \\
RC_5 & \rightarrow x^{5-1} & = x^4 & \bmod prime & = x^4 & \rightarrow 00010000 \rightarrow 10_{16} \\
RC_6 & \rightarrow x^{6-1} & = x^5 & \bmod prime & = x^5 & \rightarrow 00100000 \rightarrow 20_{16} \\
RC_7 & \rightarrow x^{7-1} & = x^6 & \bmod prime & = x^6 & \rightarrow 01000000 \rightarrow 40_{16} \\
RC_8 & \rightarrow x^{8-1} & = x^7 & \bmod prime & = x^7 & \rightarrow 10000000 \rightarrow 80_{16} \\
RC_9 & \rightarrow x^{9-1} & = x^8 & \bmod prime & = x^4 + x^3 + x + 1 & \rightarrow 00011011 \rightarrow 1B_{16} \\
RC_{10} & \rightarrow x^{10-1} & = x^9 & \bmod prime & = x^5 + x^4 + x^2 + x & \rightarrow 00110110 \rightarrow 36_{16}
\end{array}
$$

Pseudocode for key expansion in AES-128

```
KeyExpansion ([key0 to key15], [w0 to w43])
{
    for (i = 0 to 3)
        wi ← key4i + key4i+1 + key4i+2 + key4i+3

    for (i = 4 to 43)
    {
        if (i mod 4 ≠ 0)    wi ← wi–1 + wi–4
        else
        {
            t ← SubWord (RotWord (wi–1)) ⊕ RConi/4        // t is a temporary word
            wi ← t + wi–4
        }
    }
}
```



Cipher and inverse cipher of the original design

```
Cipher (InBlock [16], OutBlock[16], w[0 … 43])
{
    BlockToState (InBlock, S)

    S ← AddRoundKey (S, w[0…3])
     for (round = 1 to 10)
     {
         S ← SubBytes (S)
         S ← ShiftRows (S)
          if (round ≠ 10)   S ← MixColumns (S)
          S ← AddRoundKey (S, w[4 × round, 4 × round + 3])
      }

   StateToBlock (S, OutBlock);
}
```

Pseudocode for cipher in the original design

ANALYSIS OF AES

- Security
  - AES was designed after DES. Most of the known attacks on DES were already tested on AES; none of them has broken the security of AES so far.
- Brute-Force Attack
  - For DES we need 2^{56}(ignoring the key complement issue) tests to find the key;
  - For AES we need 2^{128} tests to find the key.
  - This means that if we can break DES in t seconds, we need ($2^{72} \times t$) seconds to break AES. This would be almost impossible.
  - AES provides two other versions with longer cipher keys. The lack of weak keys is another advantage of AES over DES.

ANALYSIS OF AES

- Statistical Attacks
  - The strong diffusion and confusion provided by the combination of the SubBytes, ShiftRows, and MixColumns transformations removes any frequency pattern in the plaintext.
  - Numerous tests have failed to do statistical analysis of the ciphertext.
- Differential and Linear Attacks
  - AES was designed after DES.
  - Differential and linear cryptanalysis attacks were no doubt taken into consideration.
  - There are no differential and linear attacks on AES as yet.
- Simplicity and Cost
  - The algorithms used in AES are so simple that they can be easily implemented using cheap processors and a minimum amount of memory.